Making the simulated markets real

Tec-C · FinTechathon 2026 · Review Finding A follow-up · repo file `agent/MULTICHAIN_NODES.md` · 17 October 2026

The Brain routes each decision to a market executor. A market is **live** when the vault is deployed on its chain and the Brain holds a session key for it; otherwise its executor answers **`simulated`** and returns no transaction hash. Nothing is ever faked.

Market	Chain	Executor	Today, out of the box	How it becomes real
Crypto	Ethereum (Anvil / Sepolia)	`ChainExecutor`	**Real** on local Anvil	Section 3 (or Section 6 for Sepolia)
Commodities	Arbitrum (Sepolia)	`ArbitrumCommoditiesExecutor`	Simulated	Section 4 (local) or Section 5 (public testnet)
Chinese market	FISCO BCOS	`FiscoBcosExecutor`	Simulated	Section 7 (local FISCO BCOS node)
Arbitrage	PegGuard two-leg plan	`PegGuardArbitrageExecutor`	Simulated (plan only)	Section 9 (needs a perp venue + repo facility)

Every live market runs the **same** vault (Member 1's `TetherGuardHook`, `SessionKeyValidator`, `TecAccount`) and the **same** executor path: mandate check → 117-byte session signature verified by the validator → dry run through the hook → signed transaction → Morning Review record. Arbitrum and FISCO BCOS can do this because both run Ethereum-compatible contracts and JSON-RPC.

---

1. One-time setup

```
brew install foundry                    # forge, anvil, cast (macOS; Linux: curl -L
https://foundry.paradigm.xyz | bash)
cd contracts && forge build && cd ..    # compiles Member 1's contracts into contracts/out
cd agent
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt -r requirements-dev.txt
.venv/bin/python -m pytest -q           # everything green before you start
```

`forge build` may rewrite `contracts/foundry.lock` (it stores a Windows path). Don't commit that change: `git checkout contracts/foundry.lock`.

2. How a market becomes live

```
.venv/bin/python scripts/deploy_market_chain.py --network <anvil|sepolia|arbitrum|fisco_bcos> --rpc
<URL>
```

The script:

1. deploys the vault suite plus mocks (USDC with 6 decimals, ETH/USD and USDC/USD feeds, one mock protocol per venue in `venues.json`), whitelists every venue with its risk tier, and sets a $500/day radius ($2,000 ceiling, elasticity on);
2. creates a fresh session key, signs an EIP-712 `UserMandate` (max $1,000 per action, 7 days, the venue list), registers it on the validator, authorises it on the hook and installs it as the account's ERC-7579 executor;
3. writes `agent/.state/markets/<network>.json` (owner-only, mode 0600; it holds the **session** key, never your deployer key).

From then on `tecc_brain.onchain.markets.build_router()` uses the live executor for that market automatically. Prove it with one real transaction:

```
.venv/bin/python scripts/deploy_market_chain.py --network <network> --smoke 25 --smoke-venue <venue_id>
# status   submitted
# tx hash  0x…            ← look it up on the chain's explorer
```

Deployer key: on any network other than `anvil`, the script reads a funded private key from an environment variable (default `TECC_DEPLOYER_KEY`). It never takes it on the command line, to keep it out of shell history. For the demo that key is also the vault's root owner (the FaceID passkey in production). Use a fresh testnet-only key.

Cost: deployment ≈ 11.1 M gas (29 transactions) and session setup ≈ 0.4 M gas (6 transactions).

3. Local Ethereum (already real)

```
anvil --port 8545                                          # terminal 1
.venv/bin/python scripts/deploy_market_chain.py --network anvil --rpc http://127.0.0.1:8545
.venv/bin/python scripts/deploy_market_chain.py --network anvil --smoke 25 --smoke-venue aave_v3_usdc
```

4. Commodities, local: a second chain with Arbitrum Sepolia's chain id

The quickest way to show two independent chains in a demo:

```
anvil --port 8547 --chain-id 421614                        # terminal 2: "Arbitrum Sepolia" stand-in
export TECC_DEPLOYER_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80  # Anvil
dev key #0, public
.venv/bin/python scripts/deploy_market_chain.py --network arbitrum --rpc http://127.0.0.1:8547
.venv/bin/python scripts/deploy_market_chain.py --network arbitrum --smoke 25 --smoke-venue paxg_hold
```

Say it honestly in the demo: this is a second **local** EVM chain using Arbitrum Sepolia's chain id. Section 5 is the public testnet.

5. Commodities on the real Arbitrum Sepolia testnet

Chain id	421614
Public RPC	`https://sepolia-rollup.arbitrum.io/rpc` (or an Alchemy / Infura Arbitrum Sepolia URL)
Explorer	`https://sepolia.arbiscan.io`
Gas needed	≈ 11.5 M gas in total; about 0.002 ETH on Arbitrum Sepolia is plenty

1. Create a new wallet (for example in MetaMask) and export its private key. Use it for testnets only.
2. Get Sepolia ETH from a faucet, then bridge a little to Arbitrum Sepolia (`bridge.arbitrum.io`, choose the Sepolia testnet), or use an Arbitrum Sepolia faucet directly.
3. Deploy and test:

```bash
export TECC_DEPLOYER_KEY=0x<your testnet key>
.venv/bin/python scripts/deploy_market_chain.py --network arbitrum --rpc https://sepolia-rollup.arbitrum.io/rpc \
  --fund-session-eth 0.001
.venv/bin/python scripts/deploy_market_chain.py --network arbitrum --smoke 25 --smoke-venue paxg_hold
```

1. Paste the tx hash into Arbiscan. You'll see `TecAccount.execute` from the session key, the hook's `TetherRadiusConsumed` and the `AutonomousExecutionLogged` record.
2. Optional: copy the addresses from `agent/.state/markets/arbitrum.json` into `shared/contracts.arbitrum.json` so the team (and Member 3's UI) sees them. Never copy the `session_key`

field.

What is real and what is mock: the vault, the session key, the signature checks, the radius and the transaction are real on the testnet. USDC, the price feeds and the "PAXG" venue are mocks deployed by the script, because testnet protocols don't match mainnet.

6. Crypto on Ethereum Sepolia

Same as Section 5 with `--network sepolia`, chain id 11155111, an RPC such as an Alchemy/Infura Sepolia URL or `https://ethereum-sepolia-rpc.publicnode.com`, and explorer `sepolia.etherscan.io`. L1 gas is pricier: at 2–10 gwei, the deployment costs 0.02–0.1 Sepolia ETH. Then set `TECC_NETWORK=sepolia` so the router uses it for the crypto market.

7. Chinese market on a FISCO BCOS node

FISCO BCOS 3.x nodes ship an **Ethereum-compatible Web3 JSON-RPC** interface, the `[web3_rpc]` section of the node's `config.ini`. That is what lets Member 1's Solidity vault and this executor run on WeBank's chain unchanged.

7.1 Start a local 4-node chain (FISCO BCOS 3.x, "Air" deployment)

Follow the official quick start for your platform (Linux or macOS), at https://fisco-bcos-doc.readthedocs.io (3.x, "Air" version). In short:

```
mkdir -p ~/fisco && cd ~/fisco
# build_chain.sh from the latest FISCO BCOS 3.x release (github.com/FISCO-BCOS/FISCO-BCOS/releases)
curl -#LO https://github.com/FISCO-BCOS/FISCO-BCOS/releases/download/<v3.x.y>/build_chain.sh && chmod
u+x build_chain.sh
bash build_chain.sh -l 127.0.0.1:4 -p 30300,20200          # 4 nodes on this machine
```

7.2 Turn on the Web3 RPC on one node

Edit `~/fisco/nodes/127.0.0.1/node0/config.ini` and add (or enable) this section. It is the documented `[web3_rpc]` section of FISCO BCOS 3.x:

```
[web3_rpc]
enable=true
listen_ip=127.0.0.1
listen_port=8545
thread_count=8
```

Use a port that isn't taken by Anvil (for example 8555 if Anvil uses 8545). Then start the chain:

```
bash ~/fisco/nodes/127.0.0.1/start_all.sh
curl -s -X POST http://127.0.0.1:8555 -H 'Content-Type: application/json' \
     -d '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

Write down the chain id it returns. The executor refuses to send if the deployment's chain id differs from the node's.

7.3 Compile for an older EVM if needed

The contracts are compiled for the `cancun` EVM. If FISCO BCOS's EVM rejects newer opcodes during deployment (errors like "invalid opcode" or "bad instruction"), build a second copy for `paris` and point the Brain at it:

```
cd contracts && forge build --evm-version paris --out out-paris && cd ../agent
export TECC_FOUNDRY_OUT=$(cd ../contracts && pwd)/out-paris
```

7.4 Deploy and test

FISCO BCOS charges no native-coin gas by default, so skip the coin transfers:

```
export TECC_DEPLOYER_KEY=0x<any secp256k1 key>            # needs no balance on a zero-gas consortium
chain
.venv/bin/python scripts/deploy_market_chain.py --network fisco_bcos --rpc http://127.0.0.1:8555 --no-
native-coin
.venv/bin/python scripts/deploy_market_chain.py --network fisco_bcos --smoke 25 --smoke-venue
shibor_mmf
```

The executor detects chains without EIP-1559 fees and signs a legacy (EIP-155) transaction. For WeBank judges, show the transaction in the node's log or with `eth_getTransactionReceipt` on the Web3 RPC.

Things to verify on your FISCO BCOS version. I couldn't test these without a node:

- that `eth_sendRawTransaction`, `eth_estimateGas` and `eth_getTransactionReceipt` are all served by `[web3_rpc]`

(the release notes mention fixes to these in recent 3.x versions, so use the newest 3.x release);

- whether an account needs a balance or permission to deploy contracts (consortium chains can restrict deployment

to authorised accounts: give your deployer address deploy permission in that case).

If FISCO BCOS can't be set up in time, keep the market as `simulated` and say "WeBank integration designed and tested on EVM chains; FISCO BCOS node planned". That is honest and still strong.

8. Using live markets in the Brain

```
from tecc_brain.onchain.markets import build_router
router = build_router()          # live where agent/.state/markets/<network>.json exists, simulated
elsewhere
result = router.submit(intent, account, market="commodities")
result.status                    # "submitted" (real tx_hash) | "simulated" (no hash) | "failed"
(reason, nothing sent)
```

The cross-chain daily limit counts only `submitted` transactions: a simulation spends nothing.

9. PegGuard: why it stays simulated

PegGuard authorises a **two-leg** trade: buy the discounted asset, and hedge with a short perpetual (Class A) or a par-capped USDT repo on FISCO BCOS (Class B). The executor returns that plan as `simulated` because neither leg has a live venue yet. To make it real you would need:

- **Class A**: a perpetuals exchange with a testnet (for example GMX v2 on Arbitrum Sepolia), an adapter contract the vault can whitelist, and the spot leg through the existing executor;
- **Class B**: a repo contract on FISCO BCOS that lends USDT capped at par, which would need to be written and audited.

Both are beyond the Oct 20 deadline. Present PegGuard as the decision engine (its maths and vetoes are real and tested) with simulated settlement.

10. Security checklist

- Testnet keys only. The deployer key lives in an environment variable and is never stored by the scripts.
- `agent/.state/` is git-ignored and holds the Brain's session keys (mode 0600). Never commit it or paste it.
- The session key can only act inside the mandate (max per action, expiry, venue list) and the vault's daily radius.

The owner can revoke it any time (`SessionKeyValidator.revokeSessionKey`, through the account).

- The hook meters ERC-20 transfers by **token** address and doesn't check the recipient (proposed fix for Member 1: require the recipient or spender of `transfer` / `approve` / `transferFrom` to be whitelisted on the AI path).

11. Troubleshooting

Message	Cause and fix
`connected to chain X, but the deployment is for chain Y`	Wrong `--rpc` or a stale `.state/markets` file: redeploy or fix the URL
`refused by the vault: TetherRadiusExceeded(...)`	Working as intended: the amount exceeds today's radius
`refused by the vault: UnauthorizedTarget` / `AssetTierNotPermitted`	Venue not whitelisted, or tier above the account's
`session signature rejected by the validator`	Session key expired, revoked, or a different mandate: redeploy (Section 2)
`set TECC_DEPLOYER_KEY ...`	Export a funded key (Sections 4–7)
`insufficient funds`	Fund the deployer, or lower `--fund-session-eth` / `--fund-account-eth`
`invalid opcode` during deployment	The chain's EVM is older: use Section 7.3
`RPC must be local ... or https`	Remote nodes must use HTTPS; tunnel plain-HTTP nodes (`ssh -L 8555:127.0.0.1:8555 host`)